

Table of Contents

[STDFDefTopic](#)

[strecords.3.01](#)

[strecords.3.02](#)

[strecords.3.03](#)

[strecords.3.04](#)

[strecords.3.05](#)

[strecords.3.06](#)

[strecords.3.07](#)

[strecords.3.08](#)

[strecords.3.09](#)

[strecords.3.10](#)

[strecords.3.11](#)

[strecords.3.12](#)

[strecords.3.13](#)

[strecords.3.14](#)

[strecords.3.15](#)

[strecords.3.16](#)

[strecords.3.17](#)

[strecords.3.18](#)

[strecords.3.19](#)

[strecords.3.20](#)

[strecords.3.21](#)

[strecords.3.22](#)

[strecords.3.23](#)

[strecords.3.24](#)

[strecords.3.25](#)

[strecords.3.26](#)

[strecords.3.27](#)

[strecords.3.28](#)

[strecstru.2.1](#)[strecstru.2.2](#)[strecstru.2.3](#)[strecstru.2.4](#)[strecstru.2.5](#)[strecstru.2.6](#)[strecstru.2.7](#)[strecstru.2.8](#)[strecstru.2.9](#)

STDFDefTopic



About STDF

About STDF

The Standard Test Data Format (STDF) defines a format for test result data. Originally developed by Teradyne, it is now an industry standard.

Note that IG-XL supports the following specific version of STDF:

Version 4, December 1986

Other STDF versions are also labeled as Version 4. IG-XL supports the Version 4 that is dated December 1986.

Note also that IG-XL does not implement the full STDF specification. When writing test result data, it uses only some of the records and only some of the fields of those records. The help section on "IG-XL Support for STDF" contains full details on how IG-XL writes data to the specific records. See IG-XL Support for STDF.

The rest of this STDF document contains these sections:

•	STDF Record Structure
---	---------------------------------------

•	STDF Record Types
---	-----------------------------------

IG-XL also supports an [ASCII version of STDF](#), called the [ASCII Test Data Format](#). See [About ATDF](#) in the section on the ATDF specification.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

strecords.3.01

<	>
-----------------	-----------------

STDF Record Types

STDF Record Types

This section contains the definitions for STDF record types. The following information is provided for each record type:

- A statement of function: how the record type is used in the STDF file.
- A table defining the data fields: first the standard STDF header, then the fields specific to this record type. The information includes the field name, the data type (see [Data Type Codes and Representation](#)), a brief description of the field, and the flag to indicate missing or invalid data (see [Optional Fields and Missing/Invalid Data](#)).
- Any additional notes on specific fields.
- Possible uses for this record type in data analysis reports. Note that this entry states only where the record type can be used. It is not a statement that the reports listed always use this record type, even if Teradyne has written those reports. For definitive information on how any data analysis software uses the STDF file, see the documentation for the data analysis software.
- Frequency with which the record type appears in the STDF file: for example, once per lot, once per wafer, or one per test.
- The location of the record type in the STDF file. This topic relates to the “Location” item under each record type. See [The Initial Sequence](#).

In addition, this section includes an alphabetical list of STDF data types. (See [Alphabetical Listing](#).)

In subsequent sections, record types appear grouped in categories.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

strecords.3.02

<	>
-----------------	-----------------

STDF Record Types : The Initial Sequence

The Initial Sequence

For several record types, the "Location" says that the record must appear "after the initial sequence." The phrase "initial sequence" refers to records that must appear at the beginning of an STDF file. The requirements for the initial sequence are as follows:

- Every file must contain:
 - One File Attributes Record (FAR)
 - One Master Information Record (MIR)
 - One or more Part Count Records (PCR)
 - One Master Results Record (MRR).

All other records are optional.

- The first record in the STDF file must be the File Attributes Record (FAR).
- If one or more Audit Trail Records (ATRs) are used, they must appear immediately after the FAR.
- The Master Information Record (MIR) must appear in every STDF file. Its location must be after the FAR and the ATRs (if ATRs are used).

- If the Retest Data Record (RDR) is used, it must appear immediately after the MIR.
- If one or more Site Description Records (SDRs) are used, they must appear immediately after the MIR and RDR (if the RDR is used).

Given these requirements, every STDF record must contain one of these initial sequences:

Initial Sequences								
FAR	–			MIR				
FAR	–	ATRs	–	MIR				
FAR	–			MIR	–	RDR		
FAR	–	ATRs	–	MIR	–	RDR		
FAR	–			MIR	–			SDRs
FAR	–	ATRs	–	MIR	–			SDRs
FAR	–			MIR	–	RDR	–	SDRs
FAR	–	ATRs	–	MIR	–	RDR	–	SDRs

All other STDF record types appear after the initial sequence.

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.03

<

>

STDF Record Types : Alphabetical Listing

Alphabetical Listing

In this section, STDF record types appear in order of ascending record type and record subtype codes. The following table lists the record types in alphabetical order.

STDF Record Types	
Abbreviation	Record Type
ATR	Audit Trail Record (ATR)
BPS	Begin Program Section Record (BPS)
DTR	Datalog Text Record (DTR)
EPS	End Program Section Record (EPS)
FAR	File Attributes Record (FAR)
FTR	Functional Test Record (FTR)
GDR	Generic Data Record (GDR)
HBR	Hardware Bin Record (HBR)
MIR	Master Information Record (MIR)
MPR	Multiple-Result Parametric Record (MPR)
MRR	Master Results Record (MRR)
PCR	Part Count Record (PCR)
PGR	Pin Group Record (PGR)
PIR	Part Information Record (PIR)

PLR	Pin List Record (PLR)
PMR	Pin Map Record (PMR)
PRR	Part Results Record (PRR)
PTR	Parametric Test Record (PTR)
RDR	Retest Data Record (RDR)
SBR	Software Bin Record (SBR)
SDR	Site Description Record (SDR)
TSR	Test Synopsis Record (TSR)
WCR	Wafer Configuration Record (WCR)
WIR	Wafer Information Record (WIR)
WRR	Wafer Results Record (WRR)

<	>
-----------------	-----------------

strecords.3.04

<

>

STDF Record Types : File Attributes Record (FAR)

File Attributes Record (FAR)

The FAR contains information necessary to decode STDF data contained in a file.
For more information, see [File Attributes Record \(FAR\)](#) in STDF Support for IG-XL.
Data Fields

FAR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (0)	
REC_SUB	U*1	Record sub-type (10)	
CPU_TYPE	U*1	CPU type that wrote this file	
STDF_VER	U*1	STDF version number	

Notes on Specific Fields

CPU_TYPE	Indicates which type of CPU wrote this STDF file. This information is useful for determining the CPU-dependent data representation of integer and floating point fields in the file’s records. Valid values are: 0 = DEC PDP-11 and VAX processors. F and D floating point formats will be used. G and H floating point formats will not be used.
----------	---

- 1 = Sun 1, 2, 3, and 4 computers.
- 2 = Sun 386i computers, and IBM PC, IBM PC-AT, and IBM PC-XT computers.
- 3-127 = Reserved for future use by Teradyne.
- 128-255 = Reserved for use by customers.

A code defined here may also be valid for other CPU types whose data formats are fully compatible with that of the type listed here. Before using one of these codes for a CPU type not listed here, check with the Teradyne hotline, which can provide additional information on CPU compatibility.

STDF_VER

Identifies the version number of the STDF specification used in generating the data. This allows data analysis programs to handle STDF specification enhancements.

Location

Required as the first record of the file.

			
	2015 Teradyne, Inc. All rights reserved.		

strecords.3.05

<

>

STDF Record Types : Audit Trail Record (ATR)

Audit Trail Record (ATR)

ATRs record operations that alter the contents of an STDF file. The name of the program and all its parameters should be recorded in the ASCII field provided in this record. Typically, this record is used to track filter programs that have been applied to the data.

For more information, see [Audit Trail Record \(ATR\)](#) in STDF Support for IG-XL.

Data Fields

ATR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (0)	
REC_SUB	U*1	Record sub-type (20)	
MOD_TIM	U*4	Date and time of STDF file modification	
CMD_LINE	C*n	Command line of program	

Frequency

Optional. One for each filter or other data transformation program applied to the STDF data.

Location

Between the File Attributes Record (FAR) and the Master Information Record (MIR).

The filter program that writes the altered STDF file must write its ATR immediately after the FAR (and hence before any other ATRs that may be in the file). In this way, multiple ATRs will be in reverse chronological order.

Possible Use

Determining whether a particular filter has been applied to the data.

<		>
	2015 Teradyne, Inc. All rights reserved.	

strecords.3.06

<

>

STDF Record Types : Master Information Record (MIR)

Master Information Record (MIR)

The MIR and the Master Results Record (MRR) contain all global information that is to be stored for a tested lot of parts. Each data stream must have exactly one MIR, immediately after the File Attributes Record (FAR) and after the Audit Trail Records (ATRs), if they are used. This allows any data reporting or analysis programs access to this information in the shortest possible time. For more information, see Master Results Record (MRR) and Master Information Record (MIR) in STDF Support for IG-XL.

Data Fields

MIR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (10)	
SETUP_T	U*4	Date and time of job setup	
START_T	U*4	Date and time first part tested	
STAT_NUM	U*1	Tester station number	
MODE_COD	C*1	Test mode code (for example, prod, dev)	space
RTST_COD	C*1	Lot retest code	space

PROT_COD	C*1	Data protection code	space
BURN_TIM	U*2	Burn-in time (in minutes)	65,535
CMOD_COD	C*1	Command mode code	space
LOT_ID	C*n	Lot ID (customer specified)	
PART_TYP	C*n	Part Type (or product ID)	
NODE_NAM	C*n	Name of node that generated data	
TSTR_TYP	C*n	Tester type	
JOB_NAM	C*n	Job name (test program name)	
JOB_REV	C*n	Job (test program) revision number	length byte = 0
SBLOT_ID	C*n	Sublot ID	length byte = 0
OPER_NAM	C*n	Operator name or ID (at setup time)	length byte = 0
EXEC_TYP	C*n	Tester executive software type	length byte = 0
EXEC_VER	C*n	Tester exec software version number	length byte = 0
TEST_COD	C*n	Test phase or step code	length byte = 0
TST_TEMP	C*n	Test temperature	length byte = 0
USER_TXT	C*n	Generic user text	length byte = 0
AUX_FILE	C*n	Name of auxiliary data file	length byte = 0
PKG_TYP	C*n	Package type	length byte = 0
FAMILY_ID	C*n	Product family ID	length byte = 0
DATE_COD	C*n	Date code	length byte = 0
FACIL_ID	C*n	Test facility ID	length byte = 0
FLOOR_ID	C*n	Test floor ID	length byte = 0
PROC_ID	C*n	Fabrication process ID	length byte = 0
OPER_FRQ	C*n	Operation frequency or step	length byte = 0
SPEC_NAM	C*n	Test specification name	length byte = 0
SPEC_VER	C*n	Test specification version number	length byte = 0

FLOW_ID	C*n	Test flow ID	length byte = 0
SETUP_ID	C*n	Test setup ID	length byte = 0
DSGN_REV	C*n	Device design revision	length byte = 0
ENG_ID	C*n	Engineering lot ID	length byte = 0
ROM_COD	C*n	ROM code ID	length byte = 0
SERL_NUM	C*n	Tester serial number	length byte = 0
SUPR_NAM	C*n	Supervisor name or ID	length byte = 0

Notes on Specific Fields

MODE_COD	Indicates the station mode under which the parts were tested. Currently defined values for the MODE_COD field are: • A = AEL (Automatic Edge Lock) mode • C = Checker mode • D = Development / Debug test mode • E = Engineering mode (same as Development mode) • M = Maintenance mode • P = Production test mode • Q = Quality Control All other alphabetic codes are reserved for future use by Teradyne. The characters 0 - 9 are available for customer use.
RTST_COD	Indicates whether the lot of parts has been previously tested under the same test conditions. Suggested values are: • Y = Lot was previously tested.

- N = Lot has not been previously tested.
- space = Not known if lot has been previously tested.
- 0 - 9 = Number of times lot has previously been tested.

PROT_COD	User-defined field indicating the protection desired for the test data being stored. Valid values are the ASCII characters 0 - 9 and A - Z. A space in this field indicates a missing value (default protection).
CMOD_COD	Indicates the command mode of the tester during testing of the parts. The user or the tester executive software defines command mode values. Valid values are the ASCII characters 0 - 9 and A - Z. A space indicates a missing value.
TEST_COD	A user-defined field specifying the phase or step in the device testing process.
TST_TEMP	The test temperature is an ASCII string. Therefore, it can be stored as degrees Celsius, Fahrenheit, Kelvin or whatever. It can also be expressed in terms like HOT, ROOM, and COLD if that is preferred.

Frequency

Always required. One per data stream.

Location

Immediately after the File Attributes Record (FAR) and the Audit Trail Records (ATR), if ATRs are used.

Possible Use

Header information for all reports

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.07

<

>

STDF Record Types : Master Results Record (MRR)

Master Results Record (MRR)

The MRR is a logical extension of the Master Information Record (MIR). The data can be thought of as belonging with the MIR, but it is not available when the tester writes the MIR information. Each data stream must have exactly one MRR as the last record in the data stream.

For more information, see Master Information Record (MIR) and Master Results Record (MRR) in STDF Support for IG-XL.

Data Fields

MRR Data Fields

ID Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (20)	
FINISH_T	U*4	Date and time last part tested	
DISP_COD	C*1	Lot disposition code	space
USR_DESC	C*n	Lot description supplied by user	length byte = 0
EXC_DESC	C*n	Lot description supplied by exec	length byte = 0

Notes on Specific Fields

DISP_COD

Supplied by the user to indicate the disposition of the lot of parts (or of the tester itself, in the case of checker or AEL data). The meaning of DISP_COD values are user-defined. A valid value is an ASCII alphanumeric character (0 - 9 or A - Z). A space indicates a missing value.

Frequency

Exactly one MRR required per data stream.

Location

Must be the last record in the data stream.

Possible Use

Final Summary Sheet, Merged Summary Sheet, Datalog, Test Results Synopsis Report, Wafer Map, Trend Plot, Histogram, ADART, Correlation, RTBM, Shmoo Plot, User Data, Repair Report

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.08

<

>

STDF Record Types : Part Count Record (PCR)

Part Count Record (PCR)

The PCR contains the part count totals for one or all test sites. Each data stream must have at least one PCR to show the part count.

For more information, see [Part Count Record \(PCR\)](#) in STDF Support for IG-XL.

Data Fields

PCR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (30)	
HEAD_NUM	U*1	Test head number	See note
SITE_NUM	U*1	Test site number	
PART_CNT	U*4	Number of parts tested	
RTST_CNT	U*4	Number of parts retested	4,294,967,295
ABRT_CNT	U*4	Number of abortions during testing	4,294,967,295
GOOD_CNT	U*4	Number of good (passed) parts tested	4,294,967,295
FUNC_CNT	U*4	Number of functional parts tested	4,294,967,295

Notes on Specific Fields

HEAD_NUM	If this PCR contains a summary of the part counts for all test sites, this field must be set to 255.
GOOD_CNT, FUNC_CNT	A part is considered good when it is binned into one of the “passing” hardware bins. A part is considered functional when it is good enough to test, whether it passes or not. Parts that are incomplete or have shorts or opens are considered nonfunctional.

Frequency

There must be at least one PCR in the file: either one summary PCR for all test sites (HEAD_NUM = 255), or one PCR for each head/site combination, or both.

Location

Anywhere in the data stream after the initial sequence (see The Initial Sequence) and before the MRR. When data is being recorded in real time, this record will usually appear near the end of the data stream.

Possible Use

Final Summary Sheet, Merged Summary Sheet, Site Summary Sheet, Report for Lot Tracking System

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.09

<

>

STDF Record Types : Hardware Bin Record (HBR)

Hardware Bin Record (HBR)

The HBR stores a count of parts “physically” placed in a particular bin after testing. (In wafer testing, “physical” binning is not an actual transfer of the chip, but rather is represented by a drop of ink or an entry in a wafer map file.) This bin count can be for a single test site (when multisite testing) or a total for all test sites. The STDF specification also supports a Software Bin Record (SBR) for logical binning categories. A part is “physically” placed in a hardware bin after testing. A part can be logically associated with a software bin during or after testing.

For more information, see [Software Bin Record \(SBR\)](#) and [Hardware Bin Record \(HBR\)](#) in STDF Support for IG-XL.

Data Fields

HBR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (40)	
HEAD_NUM	U*1	Test head number	See note
SITE_NUM	U*1	Test site number	
HBIN_NUM	U*2	Hardware bin number	
HBIN_CNT	U*4	Number of parts in bin	
HBIN_PF	C*1	Pass/fail indication	space

HBIN_NAM	C*n	Name of hardware bin	length byte = 0
----------	-----	----------------------	-----------------

Notes on Specific Fields

HEAD_NUM	If this HBR contains a summary of the hardware bin counts for all test sites, this field must be set to 255.
HBIN_NUM	Has legal values in the range 0 to 32767.
HBIN_PF	This field indicates whether the hardware bin was a passing or failing bin. Valid values: - P = Passing bin - F = Failing bin - space = Unknown

Frequency

One per hardware bin for each site. One per hardware bin for bin totals. Can be included to name unused bins.

Location

Anywhere in the data stream after the initial sequence (see The Initial Sequence) and before the MRR. When data is being recorded in real time, this record usually appears near the end of the data stream.

Possible Use

Final Summary Sheet, Merged Summary Sheet, Site Summary Sheet, Report for Lot Tracking System

< >	
	2015 Teradyne, Inc. All rights reserved.

strecords.3.10

<

>

STDF Record Types : Software Bin Record (SBR)

Software Bin Record (SBR)

The SBR stores a count of parts associated with a particular logical bin after testing. This bin count can be for a single test site (when multisite testing) or a total for all test sites. The STDF specification also supports a Hardware Bin Record (HBR) for physical binning. A part is “physically” placed in a hardware bin after testing. A part can be logically associated with a software bin during or after testing.

For more information, see [Hardware Bin Record \(HBR\)](#) and [Software Bin Record \(SBR\)](#) in [STDF Support for IG-XL](#).

Data Fields

SBR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (50)	
HEAD_NUM	U*1	Test head number	See note
SITE_NUM	U*1	Test site number	
SBIN_NUM	U*2	Software bin number	
SBIN_CNT	U*4	Number of parts in bin	
SBIN_PF	C*1	Pass/fail indication	space
SBIN_NAM	C*n	Name of software bin	length byte = 0

Notes on Specific Fields

HEAD_NUM	If this SBR contains a summary of the software bin counts for all test sites, this field must be set to 255.
SBIN_NUM	Has legal values in the range 0 to 32767.
SBIN_PF	This field indicates whether the software bin was a passing or failing bin. Valid values for this field are: - P = Passing bin - F = Failing bin - space = Unknown

Frequency

One per software bin for each site. One per software bin for bin totals. Can be included to name unused bins.

Location

Anywhere in the data stream after the initial sequence (see The Initial Sequence) and before the MRR. When data is being recorded in real time, this record usually appears near the end of the data stream.

Possible Use

Final Summary Sheet, Merged Summary Sheet, Site Summary Sheet, Report for Lot Tracking System

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

strecords.3.11

<

>

STDF Record Types : Pin Map Record (PMR)

Pin Map Record (PMR)

The PMR provides indexing of tester channel names and maps them to physical and logical pin names. Each PMR defines information for a single channel/pin combination.

For more information, see [Using Pin Mapping Records](#) and [Pin Map Record \(PMR\)](#) in STDF Support for IG-XL.

Data Fields

PMR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (60)	
PMR_INDX	U*2	Unique index associated with pin	
CHAN_TYP	U*2	Channel type	0
CHAN_NAM	C*n	Channel name	length byte = 0
PHY_NAM	C*n	Physical name of pin	length byte = 0
LOG_NAM	C*n	Logical name of pin	length byte = 0
HEAD_NUM	U*1	Head number associated with channel	1

SITE_NUM	U*1	Site number associated with channel	1
----------	-----	-------------------------------------	---

Notes on Specific Fields

PMR_INDXX	This number is used to associate channel and pin name information with data in the FTR or MPR. Reporting programs can then look up the PMR index and choose which of the three names to use. The range of valid PMR indexes is 1 - 32,767. The size of the FAIL_PIN and SPIN_MAP arrays in the FTR are directly proportional to the highest PMR index number. Therefore, start PMR indexes with a low number and use consecutive numbers if possible.
CHAN_TYX	The channel type values are tester-specific. Refer to the tester documentation for a list of the valid tester channel types and codes.
HEAD_NUM, SITE_NUM	If a test system does not support multisite testing and does not have a standard way to identify its single test site or head, set these fields to 1. If missing, the value of these fields will default to 1.

Frequency

One per channel/pin combination used in the test program. Reuse of a PMR index number is not permitted.

Location

After the initial sequence (see The Initial Sequence) and before the first PGR, PLR, FTR, or MPR that uses this record’s PMR_INDXX value.

Possible Use

Functional Datalog, Functional Histogram

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.12

<

>

STDF Record Types : Pin Group Record (PGR)

Pin Group Record (PGR)

The PGR associates a name with a group of pins.

For more information, see [Using Pin Mapping Records](#) and [Pin Group Record \(PGR\)](#) in STDF Support for IG-XL.

Data Fields

PGR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (62)	
GRP_INDX	U*2	Unique index associated with pin group	
GRP_NAM	C*n	Name of pin group	length byte = 0
INDX_CNT	U*2	Count (k) of PMR indexes	
PMR_INDX	kxU*2	Array of indexes for pins in the group	INDX_CNT = 0

Notes on Specific Fields

GRP_INDX	The range of valid group index numbers is 32,768 - 65,535.
----------	--

INDX_CNT,
PMR_INDX

PMR_INDX is an array of PMR indexes whose length is defined by INDX_CNT. The order of the PMR indexes should be from most significant to least significant bit in the pin group (regardless of the order of PMR index numbers).

Frequency

One per pin group defined in the test program.

Location

After all the PMRs whose PMR index values are listed in the PMR_INDX array of this record; and before the first PLR that uses this record's GRP_INDX value.

Possible Use

Functional Datalog

<		>	
	2015 Teradyne, Inc. All rights reserved.		

strecords.3.13

<

>

STDF Record Types : Pin List Record (PLR)

Pin List Record (PLR)

The PLR defines the current display radix and operating mode for a pin or pin group.
For more information, see [Using Pin Mapping Records](#) and [Pin List Record \(PLR\)](#) in [STDF Support for IG-XL](#).
Data Fields

PLR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (63)	
GRP_CNT	U*2	Count (k) of pins or pin groups	
GRP_INDX	kxU*2	Array of pin or pin group indexes	
GRP_MODE	kxU*2	Operating mode of pin group	0
GRP_RADX	kxU*1	Display radix of pin group	0
PGM_CHAR	kxC*1	Program state encoding characters	0
RTN_CHAR	kxC*1	Return state encoding characters	0

PGM_CHAL	kxC*1	Program state encoding characters	0
RTN_CHAL	kxC*1	Return state encoding characters	0

Notes on Specific Fields

GRP_CNT	Defines the number of pins or pin groups whose radix and mode are being defined. Therefore, it defines the size of each of the arrays that follow in the record. GRP_CNT must be greater than zero.
GRP_MODE	The following are valid values for the pin group mode: • 00 = Unknown • 10 = Normal • 20 = SCIO (Same Cycle I/O) • 21 = SCIO Midband • 22 = SCIO Valid • 23 = SCIO Window Sustain • 30 = Dual drive (two drive bits per cycle) • 31 = Dual drive Midband • 32 = Dual drive Valid • 33 = Dual drive Window Sustain • 40 = PPMU FIMV

- 41 = PPMU FVMI

Unused pin group modes in the range of 1 through 32,767 are reserved for future use. Pin group modes 32,768 through 65,535 are available for customer use.

GRP_RADX

The following are valid values for the pin group display radix:

- 0 = Use display program default
- 2 = Display in Binary
- 8 = Display in Octal
- 10 = Display in Decimal
- 16 = Display in Hexadecimal
- 20 = Display as symbolic

PGM_CHAR, PGM_CHAL

These ASCII characters are used to display the programmed state in the FTR or MPR. Use of these character arrays makes it possible to store tester-dependent display representations in a tester-independent format. If a single character is used to represent each programmed state, then only the PGM_CHAR array need be used. If two characters represent each state, then the first (left) character is stored in PGM_CHAL and the second (right) character is stored in PGM_CHAR.

RTN_CHAR, RTN_CHAL

These ASCII characters are used to display the returned state in the FTR or MPR. Use of these character arrays makes it possible to store tester-dependent display representations in a tester-independent format. If a single character is used to represent each returned state, then only the RTN_CHAR array need be used. If two characters represent each state, then the first (left) character is stored in RTN_CHAL and the second (right) character is stored in RTN_CHAR.

Note on Missing/Invalid Data Flags

For each field, the missing/invalid data flag applies to each member of the array, not to the array as a whole. Empty arrays (or empty members of arrays) can be omitted if they occur at the end of the record. Otherwise, each array must have the number of members indicated by GRP_CNT. You can then use the field’s missing/invalid data flag to indicate which array members have no data. For example, if GRP_CNT = 3, and if PGM_CHAL contains no data (but RTN_CHAL, which appears after PGM_CHAL, does), then PGM_CHAL should be an array of three missing/invalid data flags: 0, 0, 0.

Frequency

One or more whenever the usage of a pin or pin group changes in the test program.

Location

After all the PMRs and PGRs whose PMR index values and pin group index values are listed in the GRP_INDX array of this record; and before the first FTR that references pins or pin groups whose modes are defined in this record.

Possible Use

Functional Datalog

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.14

<

>

STDF Record Types : Retest Data Record (RDR)

Retest Data Record (RDR)

The RDR signals that data in this STDF file is for retested parts. Data in this record, combined with information in the Master Information Record (MIR), tells data filtering programs what data to replace when processing retest data.

For more information, see Master Information Record (MIR) and Retest Data Record (RDR) in STDF Support for IG-XL.

Data Fields

RDR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (70)	
NUM_BINS	U*2	Number (k) of bins being retested	
RTST_BIN	kxU*2	Array of retest bin numbers	NUM_BINS = 0

Notes on Specific Fields

NUM_BINS, RTST_BIN Indicates the number of hardware bins being retested and therefore the size of the RTST_BIN array that follows. If NUM_BINS is zero, then all bins

in the lot are being retested and RTST_BIN is omitted.

The LOT_ID, SUBLOT_ID, and TEST_COD of the current STDF file should match those of the STDF file that is being retested, so the data can be merged at a later time.

Frequency

Optional. One per data stream.

Location

If this record is used, it must appear immediately after the Master Information Record (MIR).

Possible Use

Tells data filtering programs how to handle retest data.

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.15

<

>

STDF Record Types : Site Description Record (SDR)

Site Description Record (SDR)

The SDR contains configuration information for one or more test sites, connected to one test head, that compose a site group.

For more information, see [Site Description Record \(SDR\)](#) in STDF Support for IG-XL.

Data Fields

SDR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (1)	
REC_SUB	U*1	Record sub-type (80)	
HEAD_NUM	U*1	Test head number	
SITE_GRP	U*1	Site group number	
SITE_CNT	U*1	Number (k) of test sites in site group	
SITE_NUM	kxU*1	Array of test site numbers	
HAND_TYP	C*n	Handler or prober type	length byte = 0
HAND_ID	C*n	Handler or prober ID	length byte = 0
CARD_TYP	C*n	Probe card type	length byte = 0
CARD_ID	C*n	Probe card ID	length byte = 0

LOAD_TYP	C*n	Load board type	length byte = 0
LOAD_ID	C*n	Load board ID	length byte = 0
DIB_TYP	C*n	DIB board type	length byte = 0
DIB_ID	C*n	DIB board ID	length byte = 0
CABL_TYP	C*n	Interface cable type	length byte = 0
CABL_ID	C*n	Interface cable ID	length byte = 0
CONT_TYP	C*n	Handler contactor type	length byte = 0
CONT_ID	C*n	Handler contactor ID	length byte = 0
LASR_TYP	C*n	Laser type	length byte = 0
LASR_ID	C*n	Laser ID	length byte = 0
EXTR_TYP	C*n	Extra equipment type field	length byte = 0
EXTR_ID	C*n	Extra equipment ID	length byte = 0

Notes on Specific Fields

SITE_GRP	Specifies a site group number (called a station number on some testers) for the group of sites whose configuration is defined by this record. Note that this is different from the station number specified in the MIR, which refers to a software station only. The value in this field must be unique within the STDF file.
SITE_CNT, SITE_NUM	Tells how many sites are in the site group that the current SDR configuration applies to. SITE_NUM is an array of those site numbers.
_TYP fields	These are the type or model number of the interface or peripheral equipment being used for testing: HAND_TYP, CARD_TYP, LOAD_TYP, DIB_TYP, CABL_TYP, CONT_TYP, LASR_TYP, EXTR_TYP
_ID fields	These are the IDs or serial numbers of the interface or peripheral equipment being used for testing: HAND_ID, CARD_ID, LOAD_ID, DIB_ID, CABL_ID, CONT_ID, LASR_ID, EXTR_ID

Frequency

One for each site or group of sites that is differently configured.

Location

Immediately after the MIR and RDR (if an RDR is used).

Possible Use

Correlation of yield to interface or peripheral equipment

<	>	
	2015 Teradyne, Inc. All rights reserved.	

strecords.3.16

<

>

STDF Record Types : Wafer Information Record (WIR)

Wafer Information Record (WIR)

The WIR acts mainly as a marker to indicate where testing of a wafer begins for each wafer tested by the test program. The WIR and the Wafer Results Record (WRR) bracket all stored information pertaining to one tested wafer. The WIR is used only when testing at wafer probe. A WIR/WRR pair will have the same HEAD_NUM and SITE_GRP values.

For more information, see [Wafer Results Record \(WRR\)](#) and [Wafer Information Record \(WIR\)](#) in STDF Support for IG-XL.

Data Fields

WIR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (2)	
REC_SUB	U*1	Record sub-type (10)	
HEAD_NUM	U*1	Test head number	
SITE_GRP	U*1	Site group number	255
START_T	U*4	Date and time first part tested	
WAFER_ID	C*n	Wafer ID	length byte = 0

Notes on Specific Fields

SITE_GRP	Refers to the site group in the SDR. This is a means of relating the wafer information to the configuration of the equipment used to test it. If this information is not known, or the tester does not support the concept of site groups, this field should be set to 255.
WAFER_ID	Is optional, but is strongly recommended to make the resultant data files as useful as possible.

Frequency
One per wafer tested.
Location

Anywhere in the data stream after the initial sequence (see The Initial Sequence) and before the MRR. Sent before testing each wafer.

Possible Use
Wafer Summary Sheet, Datalog, Wafer Map

<	>	
	2015 Teradyne, Inc. All rights reserved.	

strecords.3.17

<

>

STDF Record Types : Wafer Results Record (WRR)

Wafer Results Record (WRR)

The WRR contains result information relating to each wafer tested by the test program. The WRR and the Wafer Information Record (WIR) bracket all stored information pertaining to one tested wafer. The WRR is used only when testing at wafer probe. A WIR/WRR pair will have the same HEAD_NUM and SITE_GRP values.

For more information, see [Wafer Information Record \(WIR\)](#) and [Wafer Results Record \(WRR\)](#) in STDF Support for IG-XL.

Data Fields

WRR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (2)	
REC_SUB	U*1	Record sub-type (20)	
HEAD_NUM	U*1	Test head number	
SITE_GRP	U*1	Site group number	255
FINISH_T	U*4	Date and time last part tested	
PART_CNT	U*4	Number of parts tested	
RTST_CNT	U*4	Number of parts retested	4,294,967,295
ABRT_CNT	U*4	Number of aborts during testing	4,294,967,295

GOOD_CNT	U*4	Number of good (passed) parts tested	4,294,967,295
FUNC_CNT	U*4	Number of functional parts tested	4,294,967,295
WAFER_ID	C*n	Wafer ID	length byte = 0
FABWF_ID	C*n	Fab wafer ID	length byte = 0
FRAME_ID	C*n	Wafer frame ID	length byte = 0
MASK_ID	C*n	Wafer mask ID	length byte = 0
USR_DESC	C*n	Wafer description supplied by user	length byte = 0
EXC_DESC	C*n	Wafer description supplied by exec	length byte = 0

Notes on Specific Fields

SITE_GRP	Refers to the site group in the SDR. This is a means of relating the wafer information to the configuration of the equipment used to test it. If this information is not known, or the tester does not support the concept of site groups, set this field to 255.
WAFER_ID	Is optional, but is strongly recommended to make the resultant data files as useful as possible. A Wafer ID in the WRR supersedes any Wafer ID found in the WIR.
FABWF_ID	Is the ID of the wafer when it was in the fabrication process. This facilitates tracking of wafers and correlation of yield with fabrication variations.
FRAME_ID	Facilitates tracking of wafers once the wafer has been through the saw step and the wafer ID is no longer readable on the wafer itself. This is an important piece of information for implementing an inkless binning scheme.

Frequency

One per wafer tested.

Location

Anywhere in the data stream after the corresponding WIR. Sent after testing each wafer.

Possible Use

Wafer Summary Sheet, Datalog, Wafer Map

<		>	
	2015 Teradyne, Inc. All rights reserved.		

strecords.3.18

<

>

STDF Record Types : Wafer Configuration Record (WCR)

Wafer Configuration Record (WCR)

The WCR contains configuration information for wafers tested by the test program. The WCR provides dimensions and orientation information for all wafers and dice in the lot. This record is used only when testing at wafer probe.

For more information, [Wafer Configuration Record \(WCR\)](#) in STDF Support for IG-XL.

Data Fields

WCR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (2)	
REC_SUB	U*1	Record sub-type (30)	
WAFR_SIZ	R*4	Diameter of wafer in WF_UNITS	0
DIE_HT	R*4	Height of die in WF_UNITS	0
DIE_WID	R*4	Width of die in WF_UNITS	0
WF_UNITS	U*1	Units for wafer and die dimensions	0
WF_FLAT	C*1	Orientation of wafer flat	space
CENTER_X	I*2	X coordinate of center die on wafer	-32768

CENTER_Y	I*2	Y coordinate of center die on wafer	-32768
POS_X	C*1	Positive X direction of wafer	space
POS_Y	C*1	Positive Y direction of wafer	space

Notes on Specific Fields

WF_UNITS	Has these valid values: 0 = Unknown units 1 = Units are in inches 2 = Units are in centimeters 3 = Units are in millimeters 4 = Units are in mils
WF_FLAT	Has these valid values: - U = Up - D = Down - L = Left - R = Right - space = Unknown
CENTER_X, CENTER_Y	Use the value -32768 to indicate that the field is invalid.
POS_X	Has these valid values:

- L = Left
- R = Right
- space = Unknown

POS_Y

Has these valid values:

- U = Up
- D = Down
- space = Unknown

Frequency

One per STDF file (used only if wafer testing).

Location

Anywhere in the data stream after the initial sequence (see The Initial Sequence), and before the MRR.

Possbile Use

Wafer Map

<		>	
	2015 Teradyne, Inc. All rights reserved.		

strecords.3.19

<

>

STDF Record Types : Part Information Record (PIR)

Part Information Record (PIR)

The PIR acts as a marker to indicate where testing of a part begins for each part tested by the test program. The PIR and the Part Results Record (PRR) bracket all stored information pertaining to one tested part.

For more information, see [Part Results Record \(PRR\)](#) and [Part Information Record \(PIR\)](#) in STDF Support for IG-XL.

Data Fields

PIR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (5)	
REC_SUB	U*1	Record sub-type (10)	
HEAD_NUM	U*1	Test head number	
SITE_NUM	U*1	Test site number	

Notes on Specific Fields

HEAD_NUM, SITE_NUM	If a test system does not support multisite testing and does not have a standard way to identify its single test site or head, set these fields to 1. When multisite testing, these fields are used to associate individual datalogged results (FTRs and PTRs) with a PIR/PRR pair. An FTR or PTR
-----------------------	---

belongs to the PIR/PRR pair having the same values for HEAD_NUM and SITE_NUM.

Frequency

One per part tested.

Location

Anywhere in the data stream after the initial sequence (see The Initial Sequence), and before the corresponding PRR. Sent before testing each part.

Possible Use

Datalog

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.20

<

>

STDF Record Types : Part Results Record (PRR)

Part Results Record (PRR)

The PRR contains result information relating to each part tested by the test program. The PRR and the Part Information Record (PIR) bracket all stored information pertaining to one tested part. For more information, see [Part Information Record \(PIR\)](#) and [Part Results Record \(PRR\)](#) in STDF Support for IG-XL.

Data Fields

PRR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (5)	
REC_SUB	U*1	Record sub-type (20)	
HEAD_NUM	U*1	Test head number	
SITE_NUM	U*1	Test site number	
PART_FLG	B*1	Part information flag	
NUM_TEST	U*2	Number of tests executed	
HARD_BIN	U*2	Hardware bin number	
SOFT_BIN	U*2	Software bin number	65535
X_COORD	I*2	(Wafer) X coordinate	-32768
Y_COORD	I*2	(Wafer) Y coordinate	-32768

TEST_T	U*4	Elapsed test time in milliseconds	0
PART_ID	C*n	Part identification	length byte = 0
PART_TXT	C*n	Part description text	length byte = 0
PART_FIX	B*n	Part repair information	length byte = 0

Notes on Specific Fields

HEAD_NUM, SITE_NUM	If a test system does not support multisite testing and does not have a standard way to identify its single test site or head, set these fields to 1. In multisite test, these fields are used to associate individual datalogged results (FTRs and PTRs) with a PIR/PRR pair. An FTR or PTR belongs to the PIR/PRR pair having the same values for HEAD_NUM and SITE_NUM.
X_COORD, Y_COORD	Have legal values in the range -32767 to 32767. A missing value is indicated by the value -32768.
X_COORD, Y_COORD, PART_ID	Are all optional, but you should provide either the PART_ID or the X_COORD and Y_COORD to make the resultant data useful for analysis.
PART_FLG	Consists of eight bits, with the following meaning assigned to the value of each bit:
PART_FLG, bit 0	0 = This is a new part. Its data device does not supersede that of any previous device. 1 = The PIR, PTR, MPR, FTR, and PRR records that make up the current sequence (identified as having the same HEAD_NUM and SITE_NUM) supersede any previous sequence of records with the same PART_ID. (A repeated part sequence usually indicates a mistested part.) Note: Either Bit 0 or Bit 1 can be set, but not both. (It is also valid to have neither set.)
PART_FLG, bit 1	0 = This is a new part. Its data device does not supersede that of any previous device.

- 1 = The PIR, PTR, MPR, FTR, and PRR records that make up the current sequence (identified as having the same HEAD_NUM and SITE_NUM) supersede any previous sequence of records with the same X_COORD and Y_COORD. (A repeated part sequence usually indicates a mistested part.)

Note: Either Bit 0 or Bit 1 can be set, but not both. (It is also valid to have neither set.)

PART_FLG, bit 2

- 0 = Part testing completed normally
- 1 = Abnormal end of testing

PART_FLG, bit 3

- 0 = Part passed
- 1 = Part failed

PART_FLG, bit 4

- 0 = Pass/fail flag (bit 3) is valid
- 1 = Device completed testing with no pass/fail indication (that is, bit 3 is invalid)

PART_FLG, bits 5-7

Reserved for future use — must be 0

HARD_BIN

Has legal values in the range 0 to 32767.

SOFT_BIN

Has legal values in the range 0 to 32767. A missing value is indicated by the value 65535.

PART_FIX

This is an application-specific field for storing device repair information. It can be used for bit-encoded, integer, floating point, or character information. Regardless of the information stored, the first byte must contain the number of bytes to follow. This field can be decoded only by an application-specific analysis program. See Storing Repair Information.

Frequency

One per part tested.

Location

Anywhere in the data stream after the corresponding PIR and before the MRR. Sent after completion of testing each part.

Possible Use

Datalog, Wafer Map, RTBM, Shmoo Plot, Repair Data

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

strecords.3.21

<

>

STDF Record Types : Test Synopsis Record (TSR)

Test Synopsis Record (TSR)

The TSR contains test execution and failure counts for one parametric or functional test in the test program. Also contains static information, such as test name. The TSR is related to the Functional Test Record (FTR), the Parametric Test Record (PTR), and the Multiple Parametric Test Record (MPR) by test number, head number, and site number.

For more information, see Test Synopsis Record (TSR) in STDF Support for IG-XL.

Data Fields

TSR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (10)	
REC_SUB	U*1	Record sub-type (30)	
HEAD_NUM	U*1	Test head number	See note
SITE_NUM	U*1	Test site number	
TEST_TYP	C*1	Test type	space
TEST_NUM	U*4	Test number	
EXEC_CNT	U*4	Number of test executions	4,294,967,295
FAIL_CNT	U*4	Number of test failures	4,294,967,295
ALRM_CNT	U*4	Number of alarmed tests	4,294,967,295
TEST_NAM	C*n	Test name	length byte = 0

SEQ_NAME	C*n	Sequencer (program segment/flow) name	length byte = 0
TEST_LBL	C*n	Test label or text	length byte = 0
OPT_FLAG	B*1	Optional data flag	See note
TEST_TIM	R*4	Average test execution time in seconds	OPT_FLAG bit 2 = 1
TEST_MIN	R*4	Lowest test result value	OPT_FLAG bit 0 = 1
TEST_MAX	R*4	Highest test result value	OPT_FLAG bit 1 = 1
TST_SUMS	R*4	Sum of test result values	OPT_FLAG bit 4 = 1
TST_SQRS	R*4	Sum of squares of test result values	OPT_FLAG bit 5 = 1

Notes on Specific Fields

HEAD_NUM	If this TSR contains a summary of the test counts for all test sites, this field must be set to 255.
TEST_TYP	Indicates what type of test this summary data is for. Valid values are: • P = Parametric test • F = Functional test • M = Multiple-result parametric test • space = Unknown
EXEC_CNT, FAIL_CNT, ALRM_CNT	Are optional, but are strongly recommended because they are needed to compute values for complete final summary sheets.
OPT_FLAG	Contains the following fields: • bit 0 set = TEST_MIN value is invalid • bit 1 set = TEST_MAX value is invalid

- bit 2 set = TEST_TIM value is invalid
- bit 3 is reserved for future use and must be 1
- bit 4 set = TST_SUMS value is invalid
- bit 5 set = TST_SQRS value is invalid
- bits 6 - 7 are reserved for future use and must be 1

OPT_FLAG is optional if it is the last field in the record.

TST_SUMS, TST_SQRS Are useful in calculating the mean and standard deviation for a single lot or when combining test data from multiple STDF files.

Frequency

One for each test executed in the test program. Can optionally be used to identify unexecuted tests.

Location

Anywhere in the data stream after the initial sequence (see The Initial Sequence) and before the MRR. When test data is being generated in real-time, these records will appear after the last PRR.

Possible Use

Final Summary Sheet, Datalog, Merged Summary Sheet, Histogram, Wafer Map, Functional Histogram

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.22

<

>

STDF Record Types : Parametric Test Record (PTR)

Parametric Test Record (PTR)

The PTR contains results of a single execution of a parametric test in the test program. The first occurrence of this record also establishes the default values for all semistatic information about the test, such as limits, units, and scaling. The PTR is related to the Test Synopsis Record (TSR) by test number, head number, and site number.

For more information, see [Test Synopsis Record \(TSR\)](#) and [Parametric Test Record \(PTR\)](#) in STDF Support for IG-XL.

Data Fields

PTR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (15)	
REC_SUB	U*1	Record sub-type (10)	
TEST_NUM	U*4	Test number	
HEAD_NUM	U*1	Test head number	
SITE_NUM	U*1	Test site number	
TEST_FLG	B*1	Test flags (fail, alarm, etc.)	
PARM_FLG	B*1	Parametric test flags (drift, etc.)	
RESULT	R*4	Test result	TEST_FLG bit 1 = 1

TEST_TXT	C*n	Test description text or label	length byte = 0
ALARM_ID	C*n	Name of alarm	length byte = 0
OPT_FLAG	B*1	Optional data flag	See note
RES_SCAL	I*1	Test results scaling exponent	OPT_FLAG bit 0 = 1
LLM_SCAL	I*1	Low limit scaling exponent	OPT_FLAG bit 4 or 6 = 1
HLM_SCAL	I*1	High limit scaling exponent	OPT_FLAG bit 5 or 7 = 1
LO_LIMIT	R*4	Low test limit value	OPT_FLAG bit 4 or 6 = 1
HI_LIMIT	R*4	High test limit value	OPT_FLAG bit 5 or 7 = 1
UNITS	C*n	Test units	length byte = 0
C_RESFMT	C*n	ANSI C result format string	length byte = 0
C_LLMFMT	C*n	ANSI C low limit format string	length byte = 0
C_HLMFMT	C*n	ANSI C high limit format string	length byte = 0
LO_SPEC	R*4	Low specification limit value	OPT_FLAG bit 2 = 1
HI_SPEC	R*4	High specification limit value	OPT_FLAG bit 3 = 1

Notes on Specific Fields

Default Data

All data following the OPT_FLAG field has a special function in the STDF file. The first PTR for each test will have these fields filled in. These values will be the default for each subsequent PTR with the same test number: if a subsequent PTR has a value for one of these fields, it will be used instead of the default, for that one record only; if the field is blank, the default will be used. This method replaces use of the PDR in STDF V3. If the PTR is not associated with a test execution (that is, it contains only default information), bit 4 of the TEST_FLG field must be set, and the PARM_FLG field must be zero.

Unless the default is being overridden, omit the default data fields to save space in the file.

Note that RES_SCAL, LLM_SCAL, HLM_SCAL, UNITS, C_RESFMT, C_LLMFMT, and C_HLMFMT are interdependent. If you are overriding the default value of one, make sure that you also change the others to keep them consistent.

HEAD_NUM,
SITE_NUM

For character strings, you can override the default with a null value by setting the string length to 1 and the string itself to a single binary 0.

If a test system does not support multisite test and does not have a standard way to identify its single test site or head, set these fields to 1. In multisite test, these fields are used to associate individual datalogged results with a PIR/PRR pair. A PTR belongs to the PIR/PRR pair having the same values for HEAD_NUM and SITE_NUM.

TEST_FLG

Consists of eight bits, with the following meaning assigned to the value of each bit:

TEST_FLG, bit 0

- 0 = No alarm
- 1 = Alarm detected during testing

TEST_FLG, bit 1

- 0 = The value in the RESULT field is valid (see note on RESULT)
- 1 = The value in the RESULT field is not valid. This setting indicates that the test was executed, but no datalogged value was taken. You should read bits 6 and 7 of TEST_FLG to determine if the test passed or failed.

TEST_FLG, bit 2

- 0 = Test result is reliable
- 1 = Test result is unreliable

TEST_FLG, bit 3

- 0 = No timeout
- 1 = Timeout occurred

TEST_FLG, bit 4

- 0 = Test was executed

- 1 = Test not executed

TEST_FLG, bit 5

- 0 = No abort
- 1 = Test aborted

TEST_FLG, bit 6

- 0 = Pass/fail flag (bit 7) is valid
- 1 = Test completed with no pass/fail indication

TEST_FLG, bit 7

- 0 = Test passed
- 1 = Test failed

PARM_FLG

Is the parametric flag field, and consists of eight bits, with the following meaning assigned to the value of each bit:

PARM_FLG, bit 0

- 0 = No scale error
- 1 = Scale error

PARM_FLG, bit 1

- 0 = No drift error
- 1 = Drift error (unstable measurement)

PARM_FLG, bit 2

- 0 = No oscillation
- 1 = Oscillation detected

PARM_FLG, bit 3

- 0 = Measured value not high
- 1 = Measured value higher than high test limit

PARM_FLG, bit 4

- 0 = Measured value not low
- 1 = Measured value lower than low test limit

PARM_FLG, bit 5

- 0 = Test failed or test passed standard limits
- 1 = Test passed alternate limits

PARM_FLG, bit 6

- 0 = If result = low limit, then result is "fail."
- 1 = If result = low limit, then result is "pass."

PARM_FLG, bit 7

- 0 = If result = high limit, then result is "fail."
- 1 = If result = high limit, then result is "pass."

RESULT

The **RESULT** value is considered useful only if all the following bits from **TEST_FLG** and **PARM_FLG** are 0:

TEST_FLG:

- bit 0 = 0 no alarm
- bit 1 = 0 value in result field is valid
- bit 2 = 0 test result is reliable

- bit 3 = 0 no timeout
- bit 4 = 0 test was executed
- bit 5 = 0 no abort

PARM_FLG:

- bit 0 = 0 no scale error
- bit 1 = 0 no drift error
- bit 2 = 0 no oscillation

If any one of these bits is 1, then the PTR result should not be used.

ALARM_ID

If the alarm flag (bit 0 of TEST_FLG) is set, this field can contain the name or ID of the alarms that were triggered. Alarm names are tester-dependent.

OPT_FLAG

Is the Optional data flag and contains the following bits:

- bit 0 set = RES_SCAL value is invalid. The default set by the first PTR with this test number will be used.
- bit 1 reserved for future used and must be 1.
- bit 2 set = No low specification limit.
- bit 3 set = No high specification limit.
- bit 4 set = LO_LIMIT and LLM_SCAL are invalid. The default values set for these fields in the first PTR with this test number will be used.
- bit 5 set = HI_LIMIT and HLM_SCAL are invalid. The default values set for these fields in the first PTR with this test number will be

used.

- [bit 6 set = No Low Limit for this test](#) (LO_LIMIT and LLM_SCAL are invalid).
- [bit 7 set = No High Limit for this test](#) (HI_LIMIT and HLM_SCAL are invalid).

The OPT_FLAG field can be omitted if it is the last field in the record.

C_RESFMT,
C_LLMFMT,
C_HLMFMT

ANSI C format strings for use in formatting the test result and low and high limits (both test and spec). For example, "%7.2f". The format string is also known as an output specification string, as used with the printf statement. See any ANSI C reference man, or the man page on printf.

LO_SPEC, HI_SPEC

The specification limits are set in the first PTR and should never change. They use the same scaling and format strings as the corresponding test limits.

Frequency

One per parametric test execution.

Location

Under normal circumstances, the PTR can appear anywhere in the data stream after the corresponding Part Information Record (PIR) and before the corresponding Part Result Record (PRR).

In addition, to facilitate conversion from STDF V3, if the first PTR for a test contains default information only (no test results), it can appear anywhere after the initial sequence (see [The Initial Sequence](#)) and before the first corresponding PTR, but need not appear between a PIR and PRR.

Possible Use

Datalog, Histogram, Wafer Map

Storing and Displaying Parametric Test Data

The values stored as RESULT, LO_LIMIT, HI_LIMIT, LO_SPEC, and HI_SPEC are all normalized to the base unit stored as UNITS. The UNITS text string indicates base (whole) units only, with no scaling factor: for example, UNITS may be "AMPS" or "VOLTS" but never "uAMPS" or "mVOLTS". Therefore, the UNITS value provides enough information to represent the stored result or limit. In addition, because of this normalization, arithmetic can be performed directly on any values for which the UNITS fields agree.

In displaying a result or limit, however, it is sometimes desirable to use a scale other than the base units: for example, "uAMPS" rather than "AMPS". It is also desirable to indicate the precision to

which the value was measured. Scaling and precision are indicated by using additional fields. [Scaling uses the](#) RES_SCAL, LLM_SCAL and HLM_SCAL fields. The _SCAL value is an integer that indicates the power of ten of the scaling factor:

- [scaled result](#) = RESULT * (10 ** RES_SCAL)
- [scaled low limit](#) = LO_LIMIT * (10 ** LLM_SCAL)
- [scaled high limit](#) = HI_LIMIT * (10 ** HLM_SCAL)

The _SCAL value also serves as a code that indicates the prefix to be added to the UNITS value to obtain the correctly scaled units. The meaning of the codes is given in the following table, which defines the recognized values for the RES_SCAL, HLM_SCAL, and LLM_SCAL fields.

Scaling Code Values			
_SCAL value	UNITS Prefix	Meaning	Magnitude
15	f	femto	10**-15
12	p	pico	10**-12
9	n	nano	10**-9
6	u	micro	10**-6
3	m	milli	10**-3
2	%	percent	10**-2
0			10**0
-3	K	Kilo	10**3
-6	M	Mega	10**6
-9	G	Giga	10**9
-12	T	Tera	10**12

For example, if UNITS is AMPS and RES_SCAL is 6, the display units are uAMPS. To display a result or limit, the C_RESFMT, C_LLMFMT, and C_HLMFMT fields can be used. These provide an ANSI C compatible format string for displaying the result or limit. This string should provide all the information necessary to output the string with the correct precision using a format compatible with the data being collected.

For example, to store the result value "123.45 uAMPS," make the following assignments:

- **RESULT** = 123.4517*10**(-6) (where trailing "17" is rounding error)
- **RES_SCAL** = 6 (where "micro" has the code 6)
- **C_RESFMT** = %7.2f (where minimum field width is 7, precision is 2)
- **UNITS** = AMPS (the base units, to which scaling will be applied)

Again, notice that the **RESULT** and **UNITS** alone correctly represent the value, and that **RES_SCAL** and **C_RESFMT** are important only when it comes time to display the result. In this example, to display the result one would multiply **RESULT** by 10**6, display two digits to the right of the decimal point in a total field width of 7, look up the **RES_SCAL** value of "6" to determine the prefix "u", and display the **UNITS**:

123.45 uAMPS

			
2015 Teradyne, Inc. All rights reserved.			

strecords.3.23

<

>

STDF Record Types : Multiple-Result Parametric Record (MPR)

Multiple-Result Parametric Record (MPR)

The MPR contains results of a single execution of a parametric test in the test program, where that test returns multiple values. The first occurrence of this record also establishes the default values for all semistatic information about the test, such as limits, units, and scaling. The MPR is related to the Test Synopsis Record (TSR) by test number, head number, and site number. For more information, see [Test Synopsis Record \(TSR\)](#) and [Multiple-Result Parametric Record \(MPR\)](#) in STDF Support for IG-XL.

Data Fields

MPR Data Fields			
Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (15)	
REC_SUB	U*1	Record sub-type (15)	
TEST_NUM	U*4	Test number	
HEAD_NUM	U*1	Test head number	
SITE_NUM	U*1	Test site number	
TEST_FLG	B*1	Test flags (fail, alarm, etc.)	
PARM_FLG	B*1	Parametric test flags (drift, etc.)	
RTN_ICNT	U*2	Count (j) of PMR indexes	See note

RSLT_CNT	U*2	Count (k) of returned results	See note
RTN_STAT	jxN*1	Array of returned states	RTN_ICNT = 0
RTN_RSLT	kxR*4	Array of returned results	RSLT_CNT = 0
TEST_TXT	C*n	Descriptive text or label	length byte = 0
ALARM_ID	C*n	Name of alarm	length byte = 0
OPT_FLAG	B*1	Optional data flag	See note
RES_SCAL	I*1	Test result scaling exponent	OPT_FLAG bit 0 = 1
LLM_SCAL	I*1	Test low limit scaling exponent	OPT_FLAG bit 4 or 6 = 1
HLM_SCAL	I*1	Test high limit scaling exponent	OPT_FLAG bit 5 or 7 = 1
LO_LIMIT	R*4	Test low limit value	OPT_FLAG bit 4 or 6 = 1
HI_LIMIT	R*4	Test high limit value	OPT_FLAG bit 5 or 7 = 1
START_IN	R*4	Starting input value (condition)	OPT_FLAG bit 1 = 1
INCR_IN	R*4	Increment of input condition	OPT_FLAG bit 1 = 1
RTN_INDX	jxU*2	Array of PMR indexes	RTN_ICNT = 0
UNITS	C*n	Units of returned results	length byte = 0
UNITS_IN	C*n	Input condition units	length byte = 0
C_RESFMT	C*n	ANSI C result format string	length byte = 0
C_LLMFMT	C*n	ANSI C low limit format string	length byte = 0
C_HLMFMT	C*n	ANSI C high limit format string	length byte = 0
LO_SPEC	R*4	Low specification limit value	OPT_FLAG bit 2 = 1
HI_SPEC	R*4	High specification limit value	OPT_FLAG bit 3 = 1

Notes on Specific Fields

Default Data

All data beginning with the OPT_FLAG field has a special function in the STDF file. The first MPR for each test will have these fields filled in. These values will be the default for each subsequent MPR with the same test number: if a subsequent MPR has a value for one of these fields, it will be used instead of the default, for that one record only; if the field is blank, the default will be used.

If the MPR is not associated with a test execution (that is, it contains only default information), bit 4 of the TEST_FLG field must be set, and the PARM_FLG field must be zero.

Unless the default is being overridden, omit the default data fields to save space in the file.

Note that RES_SCAL, LLM_SCAL, HLM_SCAL, UNITS, C_RESFMT, C_LLMFMT, and C_HLMFMT are interdependent. If you are overriding the default value of one, make sure that you also make appropriate changes to the others to keep them consistent.

For character strings, you can override the default with a null value by setting the string length to 1 and the string itself to a single binary 0.

TEST_NUM

The test number does not implicitly increment for successive values in the result array.

HEAD_NUM,
SITE_NUM

If a test system does not support multiste testing and does not have a standard way to identify its single test site or head, set these fields to 1. In multisite test, these fields are used to associate individual datalogged results with a PIR/PRR pair. An MPR belongs to the PIR/PRR pair having the same values for HEAD_NUM and SITE_NUM.

TEST_FLG

Consists of eight bits, with the following meaning assigned to the value of each bit:

TEST_FLG, bit 0

- 0 = No alarm
- 1 = Alarm detected during testing

TEST_FLG, bit 1

Reserved for future use. Must be zero.

TEST_FLG, bit 2

- 0 = Test results are reliable
- 1 = Test results are unreliable

TEST_FLG, bit 3

- 0 = No timeout
- 1 = Timeout occurred

TEST_FLG, bit 4

- 0 = Test was executed
- 1 = Test not executed

TEST_FLG, bit 5

- 0 = No abort
- 1 = Test aborted

TEST_FLG, bit 6

- 0 = Pass/fail flag (bit 7) is valid
- 1 = Test completed with no pass/fail indication

TEST_FLG, bit 7

- 0 = Test passed
- 1 = Test failed

PARM_FLG

Is the parametric flag field, and consists of eight bits, with the following meaning assigned to the value of each bit:

PARM_FLG, bit 0

- 0 = No scale error
- 1 = Scale error

PARM_FLG, bit 1

- 0 = No drift error

- 1 = Drift error (unstable measurement)

PARM_FLG, bit 2

- 0 = No oscillation
- 1 = Oscillation detected

PARM_FLG, bit 3

- 0 = Measured value not high
- 1 = Measured value higher than high test limit

PARM_FLG, bit 4

- 0 = Measured value not low
- 1 = Measured value lower than low test limit

PARM_FLG, bit 5

- 0 = Test failed or test passed standard limits
- 1 = Test passed alternate limits

PARM_FLG, bit 6

- 0 = If result = low limit, then result is "fail."
- 1 = If result = low limit, then result is "pass."

PARM_FLG, bit 7

- 0 = If result = high limit, then result is "fail."
- 1 = If result = high limit, then result is "pass."

RTN_ICNT, RTN_INDX,
RTN_STAT

The number of element in the RTN_INDX and RTN_STAT arrays is determined by the value of RTN_ICNT. The RTN_STAT field is stored 4

bits per value. The first value is stored in the low order 4 bits of the byte. If the number of indexes is odd, the high order 4 bits of the last byte in RTN_STAT will be padded with zero. The indexes referred to in the RTN_INDX are the PMR indexes defined in the Pin Map Record (PMR). The return state codes are the same as those defined for the RTN_STAT field in the FTR.

RTN_ICNT can be omitted if it is zero and it is the last field in the record.

RSLT_CNT, RTN_RSLT

Defines the number of parametric test results in the RTN_RSLT. If this is a multiple pin measurement, and if PMR indexes will be specified, then the value of RSLT_CNT should be the same as RTN_ICNT. RTN_RSLT is an array of the parametric test result values.

RSLT_CNT can be omitted if it is zero and it is the last field in the record.

ALARM_ID

If the alarm flag (bit 0 of TEST_FLG) is set, this field can contain the name or ID of the alarms that were triggered. Alarm names are tester-dependent.

OPT_FLAG

Is the Optional Data Flag and contains the following bits:

- bit 0 set = RES_SCAL value is invalid. The default set by the first MPR with this test number will be used.
- bit 1 set = START_IN and INCR_IN are invalid.
- bit 2 set = No low specification limit.
- bit 3 set = No high specification limit.
- bit 4 set = LO_LIMIT and LLM_SCAL are invalid. The default values set for these fields in the first MPR with this test number will be used.
- bit 5 set = HI_LIMIT and HLM_SCAL are invalid. The default values set for these fields in the first MPR with this test number will be used.

- bit 6 set = No Low Limit for this test (LO_LIMIT and LLM_SCAL are invalid).
- bit 7 set = No High Limit for this test (HI_LIMIT and HLM_SCAL are invalid).

The OPT_FLAG field can be omitted if it is the last field in the record.

START_IN, INCR_IN,
UNITS_IN

For logging shmoo data, these fields specify the input conditions. START_IN is the beginning input value and INCR_IN is the increment, in UNITS_IN units. The input is applied and the output measured RSLT_CNT number of times. Values for INCR_IN can be positive or negative.

LO_LIMIT, HI_LIMIT,
UNITS

Regardless of how many test measurements are made, all must use the same limits, units, scaling, and significant digits.

C_RESFMT,
C_LLMFMT,
C_HLMFMT

ANSI C format strings for use in formatting the test result and low and high limits (both test and spec). For example, "%7.2f".The format string is also known as an output specification string, as used with the printf statement. See any ANSI C reference man, or the man page on printf.

LO_SPEC, HI_SPEC

The specification limits are set in the first MPR and should never change. They use the same scaling and format strings as the corresponding test limits.

Frequency

One per multiple-result parametric test execution.

Location

Anywhere in the data stream after the corresponding Part Information Record (PIR) and before the corresponding Part Result Record (PRR).

Possible Use

Datalog, Shmoo Plot

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.24

<

>

STDF Record Types : Functional Test Record (FTR)

Functional Test Record (FTR)

The FTR contains results of the single execution of a functional test in the test program. The first occurrence of this record also establishes the default values for all semistatic information about the test. The FTR is related to the Test Synopsis Record (TSR) by test number, head number, and site number.

For more information, see [Test Synopsis Record \(TSR\)](#) and [Functional Test Record \(FTR\)](#) in STDF Support for IG-XL.

Data Fields

FTR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (15)	
REC_SUB	U*1	Record sub-type (20)	
TEST_NUM	U*4	Test number	
HEAD_NUM	U*1	Test head number	
SITE_NUM	U*1	Test site number	
TEST_FLG	B*1	Test flags (fail, alarm, etc.)	
OPT_FLAG	B*1	Optional data flag	See note
CYCL_CNT	U*4	Cycle count of vector	OPT_FLAG bit 0 = 1

REL_VADR	U*4	Relative vector address	OPT_FLAG bit 1 = 1
REPT_CNT	U*4	Repeat count of vector	OPT_FLAG bit 2 = 1
NUM_FAIL	U*4	Number of pins with 1 or more failures	OPT_FLAG bit 3 = 1
XFAIL_AD	I*4	X logical device failure address	OPT_FLAG bit 4 = 1
YFAIL_AD	I*4	Y logical device failure address	OPT_FLAG bit 4 = 1
VECT_OFF	I*2	Offset from vector of interest	OPT_FLAG bit 5 = 1
RTN_ICNT	U*2	Count (j) of return data PMR indexes	See note
PGM_ICNT	U*2	Count (k) of programmed state indexes	See note
RTN_INDX	jxU*2	Array of return data PMR indexes	RTN_ICNT = 0
RTN_STAT	jxN*1	Array of returned states	RTN_ICNT = 0
PGM_INDX	kxU*2	Array of programmed state indexes	PGM_ICNT = 0
PGM_STAT	kxN*1	Array of programmed states	PGM_ICNT = 0
FAIL_PIN	D*n	Failing pin bitfield	length bytes = 0
VECT_NAM	C*n	Vector module pattern name	length byte = 0
TIME_SET	C*n	Time set name	length byte = 0
OP_CODE	C*n	Vector Op Code	length byte = 0
TEST_TXT	C*n	Descriptive text or label	length byte = 0
ALARM_ID	C*n	Name of alarm	length byte = 0
PROG_TXT	C*n	Additional programmed information	length byte = 0
RSLT_TXT	C*n	Additional result information	length byte = 0
PATG_NUM	U*1	Pattern generator number	255
SPIN_MAP	D*n	Bit map of enabled comparators	length byte = 0

Notes on Specific Fields

Default Data

All data starting with the PATG_NUM field has a special function in the STDF file. The first FTR for each test will have these fields filled in. These values will be the default for each subsequent FTR with the same test number. If a subsequent FTR has a value for one of these fields, it will be

used instead of the default, for that one record only. If the field is blank, the default will be used. This method replaces use of the FDR in STDF V3. Unless the default is being overridden, omit the default data fields to save space in the file.

HEAD_NUM,
SITE_NUM

If a test system does not support multisite test and does not have a standard way to identify its single test site or head, set these fields to 1. In multisite test, these fields are used to associate individual datalogged results with a PIR/PRR pair. An FTR belongs to the PIR/PRR pair having the same values for HEAD_NUM and SITE_NUM.

TEST_FLG

Consists of eight bits, with the following meaning assigned to the value of each bit:

TEST_FLG, bit 0

- 0 = No alarm
- 1 = Alarm detected during testing

TEST_FLG, bit 1

Reserved for future use. Must be zero.

TEST_FLG, bit 2

- 0 = Test result is reliable
- 1 = Test result is unreliable

TEST_FLG, bit 3

- 0 = No timeout
- 1 = Timeout occurred

TEST_FLG, bit 4

- 0 = Test was executed
- 1 = Test not executed

TEST_FLG, bit 5

- 0 = No abort

- 1 = Test aborted

TEST_FLG, bit 6

- 0 = Pass/fail flag (bit 7) is valid
- 1 = Test completed with no pass/fail indication

TEST_FLG, bit 7

- 0 = Test passed
- 1 = Test failed

OPT_FLAG

Contains the following fields:

- bit 0 set = CYCL_CNT data is invalid
- bit 1 set = REL_VADR data is invalid
- bit 2 set = REPT_CNT data is invalid
- bit 3 set = NUM_FAIL data is invalid
- bit 4 set = XFAIL_AD and YFAIL_AD data are invalid
- bit 5 set = VECT_OFF data is invalid (offset defaults to 0)
- bits 6 - 7 are reserved for future use and must be 1

This field is only optional if it is the last field in the record.

XFAIL_AD, YFAIL_AD

The logical device address produced by the memory pattern generator, before going through conversion to a physical memory address. This logical address can be different from the physical address presented to the DUT pins.

VECT_OFF	This is the integer offset of this vector (in sequence of execution) from the vector of interest (usually the failing vector). For example, if this FTR contains data for the vector before the vector of interest, this field is set to -1. If this FTR contains data for the third vector after the vector of interest, this field is set to 3. If this FTR is the vector of interest, VECT_OFF is set to 0. It is therefore possible to record an entire sequence of vectors around a failing vector for use with an offline debugger or analysis program.
RTN_ICNT, PGM_ICNT	These fields can be omitted if all data following them is missing or invalid.
RTN_ICNT, RTN_INDX, RTN_STAT	The size of the RTN_INDX and RTN_STAT arrays is determined by the value of RTN_ICNT. The RTN_STAT field is stored 4 bits per value. The first value is stored in the low order 4 bits of the byte. If the number of indexes is odd, the high order 4 bits of the last byte in RTN_STAT will be padded with zero. The indexes referred to in the RTN_INDX are those defined in the PMR.
RTN_STAT	The table of valid returned state values (expressed as hexadecimal digits) is: • 0 = 0 or low • 1 = 1 or high • 2 = midband • 3 = glitch • 4 = undetermined • 5 = failed low • 6 = failed high • 7 = failed midband

- 8 = failed with a glitch
- 9 = open
- A = short

The characters generated to represent these values are tester-dependent, and are specified in the PLR.

PGM_ICNT,
PGM_INDX,
PGM_STAT

The size of the PGM_INDX and PGM_STAT arrays is determined by the value of PGM_ICNT. The indexes referred to in the PGM_INDX are those defined in the PMR.

PGM_STAT

The tables of valid program state values (expressed in hexadecimal) are listed below. Note that there are three defined program modes: Normal, Dual Drive (two drive bits per cycle), and SCIO (same cycle I/O). The characters generated to represent these values are tester-dependent, and are specified in the PLR.

PGM_STAT
(Normal Mode
Program States)

These are the normal mode program states, with typical state representation shown in parentheses:

- 0 = Drive Low (0)
- 1 = Drive High (1)
- 2 = Expect Low (L)
- 3 = Expect High (H)
- 4 = Expect Midband (M)
- 5 = Expect Valid - not midband (V)
- 6 = Don't drive, or compare (X)

- 7 = Keep window open from prior cycle - used to "stretch" a comparison across cycles (W)

PGM_STAT

(Dual Drive Mode Program States)

These are the dual drive mode program states, with typical state representation shown in parentheses:

- 0 = Low at D2, Low at D1 times (00 0)
- 1 = Low at D2, High at D1 times (10 1)
- 2 = Hi at D2, Low at D1 times (01 2)
- 3 = Hi at D2, High at D1 times (11 3)
- 4 = Compare Low (L)
- 5 = Compare High (H)
- 6 = Compare Midband (M)
- 7 = Don't Compare (X)

PGM_STAT

(SCIO Mode Program States)

These are the SCIO mode program states, with typical state representation shown in parentheses:

- 0 = Drive Low, Compare Low (0L l)
- 1 = Drive Low, Compare High (0H h)
- 2 = Drive Low, Compare Midband (0M m)
- 3 = Drive Low, Don't Compare (0X x)
- 4 = Drive High, Compare Low (1L L)

- 5 = Drive High, Compare High (1H H)
- 6 = Drive High, Compare Midband (1M M)
- 7 = Drive High, Don't Compare (1X X)

FAIL_PIN	Encoded with PMR index 0 in bit 0 of the field, PMR index 1 in the 1st position, and so on. Bits representing PMR indexes of failing pins are set to 1.
ALARM_ID	If the alarm flag (bit 0 of TEST_FLG) is set, this field can optionally contain the name or ID of the alarm or alarms that were triggered. The names of these alarms are tester-dependent.
SPIN_MAP	This field contains an array of bits corresponding to the PMR index numbers of the enabled comparators. The 0th bit corresponds to PMR index 0, the 1st bit corresponds to PMR index 1, and so on. Each comparator that is enabled will have its corresponding PMR index bit set to 1.

Frequency

One or more for each execution of a functional test.

Location

Anywhere in the data stream after the corresponding Part Information Record (PIR) and before the corresponding Part Result Record (PRR).

Possible Use

Datalog, Functional Histogram, Functional Failure Analyzer

<

>

2015 Teradyne, Inc. All rights reserved.

strecords.3.25

<

>

STDF Record Types : [Begin Program Section Record \(BPS\)](#)

[Begin Program Section Record \(BPS\)](#)
The BPS marks the beginning of a new program section (or sequencer) in the test program.
For more information, see [Begin Program Section Record \(BPS\)](#) in STDF Support for IG-XL.
[Data Fields](#)

BPS Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (20)	
REC_SUB	U*1	Record sub-type (10)	
SEQ_NAME	C*n	Program section (or sequencer) name	length byte = 0

[Frequency](#)
Optional on each entry into the program segment.
[Location](#)
Anywhere after the PIR and before the PRR.
[Possible Use](#)
When performing analyses on a particular program segment’s test.

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

strecords.3.26

<

>

STDF Record Types : End Program Section Record (EPS)

End Program Section Record (EPS)

The EPS marks the end of the current program section (or sequencer) in the test program. For more information, see [End Program Section Record \(EPS\)](#) in STDF Support for IG-XL.

Data Fields

EPS Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (20)	
REC_SUB	U*1	Record sub-type (20)	

Note that pairs of BPS and EPS records can be nested: for example, when one sequencer calls another. In this case, the sequence of records could look like this:

BPS SEQ_NAME = sequence-1

BPS SEQ_NAME = sequence-2

EPS (end of sequence-2)

EPS (end of sequence-1)

Because an EPS record does not contain the name of the sequencer, it should be assumed that each EPS record matches the last unmatched BPS record.

Frequency

Optional on each exit from the program segment.

Location

Following the corresponding BPS and before the PRR in the data stream.

Possible Use

When performing analyses on a particular program segment’s test.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

strecords.3.27

<

>

STDF Record Types : Generic Data Record (GDR)

Generic Data Record (GDR)

The GDR contains information that does not conform to any other record type defined by the STDF specification. Such records are intended to be written under control of test programs executing on the tester. This data can be used for any purpose.

For more information, see [Generic Data Record \(GDR\)](#) in STDF Support for IG-XL.

Data Fields

GDR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (50)	
REC_SUB	U*1	Record sub-type (10)	
FLD_CNT	U*2	Count of data fields in record	
GEN_DATA	V*n	Data type code and data for one field (Repeat GEN_DATA for each data field)	

Notes on Specific Fields

GEN_DATA	Is repeated FLD_CNT number of times. Each GEN_DATA field consists of a data type code followed by the data. The data type code is the first
----------	---

unsigned byte of the field. Valid data types are listed below.

Valid Data Type Codes

The following are the valid data type codes that can be used in the GEN_DATA field:

0 = B*0	Special pad field, of length 0 (See note below)
1 = U*1	One byte unsigned integer
2 = U*2	Two byte unsigned integer
3 = U*4	Four byte unsigned integer
4 = I*1	One byte signed integer
5 = I*2	Two byte signed integer
6 = I*4	Four byte signed integer
7 = R*4	Four byte floating point number
8 = R*8	Eight byte floating point number
10 = C*n	Variable length ASCII character string (first byte is string length in bytes)
11 = B*n	Variable length binary data string (first byte is string length in bytes)
12 = D*n	Bit encoded data (first two bytes of string are length in bits)
13 = N*1	Unsigned nibble

Pad Field (Data Type 0)

Data type 0, the special pad field, is used to force alignment of following data types in the record. In particular, it must be used to ensure even byte alignment of U*2, U*4, I*2, I*4, R*4, and R*8 data types.

The GDR is guaranteed to begin on an even byte boundary. The GDR header contains four bytes. The first GEN_DATA field, therefore, begins on an even byte boundary. It is the responsibility of the designer of a GDR record to provide the pad bytes needed to ensure data boundary alignment for the CPU on which it will run.

Example

The following table describes a sample GDR that contains three data fields of different data types. The assumption is that numeric data of more than one byte must begin on an even boundary. Pad bytes will be used to meet this requirement.

GDR Example

Data	Code	Alignment Requirement
"AB"	10	A variable-length character string can begin on any byte. This field will contain one data byte, one length byte, and two data bytes, for a total length of 4 bytes. Because this field begins on an even byte, the next field also begins on an even byte.
255	1	A one-byte numeric value can begin on any byte. This field contains two bytes, so the next field also begins on an even byte.
510	5	A two-byte numeric value must begin on an even byte. This GEN_DATA field would begin on an even byte; and, because the first byte is the data code, the numeric value would begin on an odd byte. This field must therefore be preceded by a pad byte.

The byte representation for this GDR is as follows. The byte ordering shown here is for sample purposes only. The data representation differs between CPUs. The byte values are shown in hexadecimal. The decimal equivalents are given in the description of the bytes.

Bytes for GDR Example

Even Byte	Odd Byte	Description (with Decimal Values)
0c	00	Number of bytes following the header (12)
32	0a	Record type (50); record subtype (10)
04	00	Number of data fields (4)
0a	02	Character string: code (10) and length (2)
41	42	Character string: data bytes ("A" and "B")
01	ff	1-byte integer: code (1) and data (255 = 0xff)
00	05	Pad byte (0); code (5) for next field
fe	01	2-byte signed integer (510 = 0x01fe)

Frequency

A test data file can contain any number of GDRs.

Location

Anywhere in the data stream after the initial sequence (see [The Initial Sequence](#)).

Possible Use

User-written reports

< >		
	2015 Teradyne, Inc. All rights reserved.	

strecords.3.28

<

>

STDF Record Types : Datalog Text Record (DTR)

Datalog Text Record (DTR)

The DTR contains text information that is to be included in the datalog output. DTRs can be written under the control of a test program: for example, to highlight unexpected test results. They can also be generated by the tester executive software: for example, to indicate that the datalog sampling rate has changed. DTRs are placed as comments in the datalog listing.

For more information, see [Datalog Text Record \(DTR\)](#) in [STDF Support for IG-XL](#).

Data Fields

DTR Data Fields

Field Name	Data Type	Field Description	Missing/Invalid Data Flag
REC_LEN	U*2	Bytes of data following header	
REC_TYP	U*1	Record type (50)	
REC_SUB	U*1	Record sub-type (30)	
TEXT_DAT	C*n	ASCII text string	

Frequency

A test data file can contain any number of DTRs.

Location

Anywhere in the data stream after the initial sequence (see [The Initial Sequence](#)).

Possible Use

Datalog

<	>	
	2015 Teradyne, Inc. All rights reserved.	

strecstru.2.1

<	>
-----------------	-----------------

STDF Record Structure

STDF Record Structure

This section describes the basic STDF record structure. It describes the following general topics, which are applicable to all record types:

- | | |
|---|--|
| • | STDF Record Header |
| • | Record Types and Subtypes |
| • | Data Type Codes and Representation |
| • | Optional Fields and Missing/Invalid Data |
| • | STDF Filenames |
| • | STDF File Ordering |
| • | Storing Repair Information |
| • | Using Pin Mapping Records |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

strecstru.2.2

<	>
-----------------	-----------------

STDF Record Structure : STDF Record Header

STDF Record Header

Each STDF record begins with a record header consisting of three fields:

STDF Fields

Field	Description
REC_LEN	The number of bytes of data following the record header. REC_LEN does not include the four bytes of the record header.
REC_TYP	An integer identifying a group of related STDF record types.
REC_SUB	An integer identifying a specific STDF record type within each REC_TYP group. On REC_TYP and REC_SUB, see Record Types and Subtypes .

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

strecstru.2.3

<

>

STDF Record Structure : Record Types and Subtypes

Record Types and Subtypes

The header of each STDF record contains a pair of fields called REC_TYP and REC_SUB. Each REC_TYP value identifies a group of related STDF record types. Each REC_SUB value identifies a single STDF record type within a REC_TYP group. The combination of REC_TYP and REC_SUB values uniquely identifies each record type. This design allows groups of related records to be easily identified by data analysis programs, while providing unique identification for each type of record in the file.

All REC_TYP and REC_SUB codes less than 200 are reserved for future use by Teradyne. All codes greater than 200 are available for custom applications use. Codes are all in decimal values. The official list of codes and documentation for their use is maintained by Teradyne.

The following table lists the meaning of REC_TYP codes currently defined by Teradyne, as well as REC_SUB codes defined in the STDF specification.

STDF Record Types	
REC_TYP Code	Meaning and STDF REC_SUB Codes
0	Information about the STDF file 10: File Attributes Record (FAR) 20: Audit Trail Record (ATR)
1	Data collected on a per lot basis

- 10: Master Information Record (MIR)
- 20: Master Results Record (MRR)
- 30: Part Count Record (PCR)
- 40: Hardware Bin Record (HBR)
- 50: Software Bin Record (SBR)
- 60: Pin Map Record (PMR)
- 62: Pin Group Record (PGR)
- 63: Pin List Record (PLR)
- 70: Retest Data Record (RDR)
- 80: Site Description Record (SDR)

2

Data collected per wafer

- 10: Wafer Information Record (WIR)
- 20: Wafer Results Record (WRR)
- 30: Wafer Configuration Record (WCR)

5	Data collected on a per part basis
	• 10: Part Information Record (PIR) • 20: Part Results Record (PRR)
10	Data collected per test in the test program
	• 30: Test Synopsis Record (TSR)
15	Data collected per test execution
	• 10: Parametric Test Record (PTR) • 15: Multiple-Result Parametric Record (MPR) • 20: Functional Test Record (FTR)
20	Data collected per program segment
	• 10: Begin Program Section Record (BPS) • 20: End Program Section Record (EPS)
50	Generic Data
	• 10: Generic Data Record (GDR) • 30: Datalog Text Record (DTR)
180	Reserved for use by IMAGE software
181	Reserved for use by IG900 software

<		>	
	2015 Teradyne, Inc. All rights reserved.		

strecstru.2.4

<

>

STDF Record Structure : Data Type Codes and Representation

Data Type Codes and Representation

The STDF specification uses a set of concise data type codes. For example, R*4 indicates a REAL (float) value stored in four bytes. A byte consists of eight bits of data. In this document, the low order bit of each byte is designated as bit 0 and the high order bit as bit 7.

The following table lists STDF data type codes and the equivalent C language type specifier.

STDF Data Type Codes		
Code	Description	C Type Specifier
C*12	Fixed length character string: If a fixed length character string does not fill the entire field, it must be left-justified and padded with spaces.	char[12]
C*n	Variable length character string: first byte = unsigned count of bytes to follow (maximum of 255 bytes)	char[]
C*f	Variable length character string: string length is stored in another field	char[]
U*1	One byte unsigned integer	unsigned char
U*2	Two byte unsigned integer	unsigned short
U*4	Four byte unsigned integer	unsigned long
I*1	One byte signed integer	char
I*2	Two byte signed integer	short

I*4	Four byte signed integer	long
R*4	Four byte floating point number	float
R*8	Eight byte floating point number	long float (double)
B*6	Fixed length bit-encoded data	char[6]
V*n	Variable data type field: The data type is specified by a code in the first byte, and the data follows (maximum of 255 bytes)	
B*n	Variable length bit-encoded field: First byte = unsigned count of bytes to follow (maximum of 255 bytes). First data item in least significant bit of the second byte of the array (first byte is count.)	char[]
D*n	Variable length bit-encoded field: First two bytes = unsigned count of bits to follow (maximum of 65,535 bits). First data item in least significant bit of the third byte of the array (first two bytes are count). Unused bits at the high order end of the last byte must be zero.	char[]
N*1	Unsigned integer data stored in a nibble. (Nibble = 4 bits of a byte). First item in low 4 bits, second item in high 4 bits. If an odd number of nibbles is indicated, the high nibble of the byte will be zero. Only whole bytes can be written to the STDF file.	char
kxTYPE	Array of data of the type specified. The value of 'k' (the number of elements in the array) is defined in an earlier field in the record. For example, an array of short unsigned integers is defined as kxU*2.	TYPE[]

Note on Time and Date Usage

The date and time field used in this specification is defined as a four byte (32 bit) unsigned integer field measuring the number of seconds since midnight on January 1st, 1970, in the local time zone. This is the UNIX standard base time, adjusted to the local time zone.

Note on Data Representation

When data is shared among systems with different CPUs, a problem arises of little or no standardization of data representation (that is, bit ordering of various data types). Moreover, different CPUs made by the same company sometimes store data in incompatible ways. To address this problem, the STDF specification uses a field called CPU_TYPE in the File Attributes Record (FAR). This field indicates the type of processor that wrote the data. The field is used as follows:

- When writing an STDF file, a system uses its own native data representation. The type of the writing processor is stored in the CPU_TYPE field.
- When reading an STDF file, a system may need to convert records to its native data representation as it reads them. To do so, it checks the value of the CPU_TYPE field in the FAR, which is the first record in the file. Then, if the writing CPU's data representation is incompatible with its own, it uses a subroutine that reads the next (or selected) record and converts the records to its own data representation as it reads them.

This approach has the following advantages:

- All testers and hosts can read and write local data using their native data representation.
- Testing and local data analysis are not slowed by performing data conversions on any tester.
- Use of a read subroutine makes data conversion transparent at read time.

This approach works for any combination of host and tester processors, provided that the machines can store and read test data in eight bit bytes.

For more information, see File Attributes Record (FAR).

			
2015 Teradyne, Inc. All rights reserved.			

strecstru.2.5

<	>
-----------------	-----------------

STDF Record Structure : Optional Fields and Missing/Invalid Data

Optional Fields and Missing/Invalid Data

Certain fields in STDF records are defined as optional. An optional field must be present in the record, but there are ways to indicate that its value is not meaningful, that is, that its data should be considered missing or invalid. The methods are:

- Some optional fields have a predefined value that means that the data for the field is missing. For example, if the optional field is a variable-length character string, a length byte of 0 means that the data is missing. If the field is numeric, a value of -1 may be defined as meaning that the data is missing.
- For other optional fields, all possible stored values, including -1, are legal. In this case, the STDF specification for the record defines an Optional Data bit field. Each bit is used to designate whether an optional field in the record contains valid or invalid data. Usually, if the bit for an optional field is set, any data in the field is invalid and should be ignored.

Optional fields at the end of a record can be omitted to save storage space. To be omitted, an optional field must have missing or invalid data, and all fields following it must be optional fields containing missing or invalid data. It is never valid to omit an optional field from the middle of a record.

The specification of each STDF record has a column labelled Missing/Invalid Data Flag. An entry in this column means that the field is optional, and that the value shown is the way to flag the field's data as missing or invalid. If the column does not have an entry, the field is required.

Each data type has a standard way of indicating missing or invalid data.

Missing/Invalid Data Flags for Data Types

Data Type	Missing/Invalid Data Flag
Variable-length string	Set the length byte to 0.
Fixed-length character string	Fill the field with spaces.
Fixed-length binary string	Set a flag bit in an Optional Data byte.
Time and date fields	Use a binary 0.
Signed and unsigned integers and floating point values	Use the indicated reserved value or set a flag bit in an Optional Data byte.

Note on Required and Optional Fields

The distinction between required and optional fields applies only to the definition of a minimally valid STDF file. It is not a statement about whether any software (even Teradyne software) requires the field. A field that is marked optional in the specification may be required by software that reads or analyzes the STDF file, even if Teradyne has written the software.

In most cases, a minimally valid STDF file will not provide sufficient input for a piece of analysis software. You will need to fill in some fields or records that are not marked as required here.

This specification is not intended to define the data requirements for any analysis software. The only authority on whether a piece of software requires a certain STDF field or record is the documentation for that software.

strecstru.2.6

<

>

STDF Record Structure : STDF Filenames

STDF Filenames

An STDF file name must have the following format:

filename.STD[string]

where:

filename	Is any string consisting of 1 to 39 of the ASCII characters A - Z, a - z, and 0 - 9, plus the underscore (_). The first character must be alphabetic. Users should be aware that, while some operating systems distinguish between uppercase and lowercase characters, most do not.
.STD[string]	Is a string beginning with the characters .STD, and continuing with characters that are valid for filename. The string cannot be longer than 39 characters. Under systems that support file extensions, this is the file extension. Under systems that do not, it is considered to be a fixed literal string. For systems that distinguish between uppercase and lowercase characters, this string should be in lowercase (.std).

Note these points:

- Previous versions of the STDF specification allow the dollar sign (\$) as a valid filename character. It is no longer supported, because its use is incompatible with some operating systems.
- An STDF filename can contain only a single period. Software that processes STDF files may check for an extension, which is defined as the string after the first period. Many operating

systems permit only one period per filename.

- Use only characters defined as valid for the filename. This restricted set is designed to be compatible with as many operating systems and software packages as possible. Using other characters may have unforeseen consequences. For example, some data analysis software may not accept a filename containing a character that you used.
- Use only .STD, without any additional string for the extension. If you must add additional characters, add as few as possible. Software that processes STDF files may add characters to the .STD extension to indicate the state of processing. To avoid exceeding system-specific limits, the original filename extension should be as short as possible (that is, .STD).
- Some software that processes STDF files retains only the part of the filename to the left of the period (the filename part, not the .STD extension part). Therefore, the filename to the left of the period should be unique to ensure that the names remain unique after other software has processed the file.

Choose STDF file names that:

- Provide unique file names throughout a system
- Indicate the data contained in the file
- Indicate when the test data was generated
- Provide some level of customer control
- Work on a variety of computer systems



strecstru.2.7

<	>
-----------------	-----------------

STDF Record Structure : STDF File Ordering

STDF File Ordering

Test data collected by testers is usually written directly to files in STDF format. Each STDF file contains test data for one lot of parts. To make the data management software efficient and reliable, all raw test data for a single insertion of a single lot should be stored in one STDF data file. An STDF test data file must contain one FAR, one MIR, at least one PCR, and one MRR. All other records are optional. The file may, therefore, contain any combination of datalog, summary, and site summary for that lot of parts.

Data records in an STDF file can be arranged in a variety of ways. The following factors can affect the record ordering:

- | | |
|---|---|
| • | Whether wafers are being tested |
| • | Whether multisite testing is in effect |
| • | Whether test description records are being used |
| • | Whether datalogging is in effect |

The following topics show different ways in which the STDF format can be used to store test data. The cases shown are as follows:

- | | |
|---|---|
| • | For a Lot of Packaged Devices |
| • | For a Lot of Devices for which Only Summary Information is Required |

- For a Lot of Devices at Wafer Probe
- For a Lot of Devices, Storing Only Information Necessary to Generate a Wafer Map
- For a Lot of Devices Tested at Wafer Probe Using Multisite Test (two test heads, two sites per head)

For a Lot of Packaged Devices

File Ordering for Lot of Packaged Devices

Information Datalogged	Record Type Used
Global information for the file	FAR
Global information for entire lot	MIR
Testing each part:	
Information on first tested part	PIR
Results of first test on first part	PTR/MPR/FTR
More results on first part	PTR/MPR/FTR
Results of final test on first part.	PTR/MPR/FTR
Final results on first tested part	PRR
Information on second tested part	PIR
Repeat test suite for second part	PTR/MPR/FTR
Final results on second tested part	PRR
Repeat sequence for each tested part	PIR
	...
	PRR
Final results for entire lot:	
Summary (count of test executions, count of failed parts, etc.) for each test in job plan (one TSR per test in plan)	TSR
	...
	TSR
Count of parts placed in each hardware bin (one HBR per hardware bin)	HBR
	...

	HBR
Count of parts assigned to each logical bin (one SBR per software bin)	SBR
	...
	SBR
Part count totals for lot	PCR
Global summary of results for entire lot	MRR

For a Lot of Devices for which Only Summary Information is Required

File Ordering for Summary Information

Information Datalogged	Record Type Used
Global information for the file	FAR
Global information for entire lot	MIR
Final results for entire lot:	
Summary (count of test executions, count of failed parts, etc.) for each test in job plan (one TSR per test in plan)	TSR
	...
	TSR
Count of parts placed in each hardware bin (one HBR per hardware bin)	HBR
	...
	HBR
Count of parts assigned to each logical bin (one SBR per software bin)	SBR
	...
	SBR
Part count totals for lot	PCR
Global summary of results for entire lot	MRR

For a Lot of Devices at Wafer Probe

File Ordering for Wafer Probe

Information Datalogged	Record Type Used
Global information for the file	FAR
Global information for entire lot	MIR

Dimensions and orientation of wafer	WCR
Testing each wafer:	
Information for first wafer	WIR
Information for first die	PIR
Perform test suite on first die	PTR/MPR/FTR
	...
	PTR/MPR/FTR
Final results of test suite on first die	PRR
Repeat for each die of first wafer	PIR
	PTR/MPR/FTR
	...
	PTR/MPR/FTR
	PRR
Test results summary for dice of first wafer	WRR
Repeat sequence for each remaining wafer	WIR
	... WRR
Final results for entire lot:	
Summary (count of test executions, count of failed parts, etc.) for each test in job plan (one TSR per test in plan)	TSR
	...
	TSR
Count of dice placed in each hardware bin (one HBR per hardware bin)	HBR
	...
	HBR
Count of dice placed in each logical bin (one SBR per software bin)	SBR
	...
	SBR
Part count totals for lot	PCR
Global summary of results for entire lot	MRR

For a Lot of Devices, Storing Only Information Necessary to Generate a Wafer Map

File Ordering for Wafer Map

Information Datalogged	Record Type Used
Global information for the file	FAR
Global information for entire lot	MIR
Dimensions and orientation of wafer	WCR
Testing each wafer:	
Information for wafer	WIR
Information for first die	PIR
Final results of test suite on first die	PRR
Repeat for each remaining die	PIR
	PRR
	...
Test results summary for all dice of wafer	WRR
Repeat sequence for remaining wafers	WIR
	...
	WRR
Final results for entire lot:	
Summary (count of test executions, count of failed parts, etc.) for each test in job plan (one TSR per test in plan)	TSR
	...
	TSR
Count of dice placed in each hardware bin (one HBR per hardware bin)	HBR
	...
	HBR
Count of dice placed in each logical bin (one SBR per software bin)	SBR
	...
	SBR
Part count totals for lot	PCR
Global summary of results for entire lot	MRR

For a Lot of Devices Tested at Wafer Probe Using Multisite Test (two test heads, two sites per head)

File Ordering for Wafer Probe for Multisite Test

Information Datalogged	Record Type Used
Global information for the file	FAR
Global information for entire lot	MIR
Dimensions and orientation of wafer	WCR
Testing first two wafers (one per test head):	
Information for first wafer (head 1)	WIR, head 1
Information for second wafer (head 2)	WIR, head 2
Beginning of first die	PIR, head 1 site 1
Beginning of second die	PIR, head 1 site 2
Beginning of third die	PIR, head 2 site 1
Beginning of fourth die	PIR, head 2 site 2
First test on first die	PTR/MPR/FTR, head 1 site 1
First test on second die	PTR/MPR/FTR, head 1 site 2
First test on third die	PTR/MPR/FTR, head 2 site 1
First test on fourth die	PTR/MPR/FTR, head 2 site 2
Second test on first die	PTR/MPR/FTR, head 1 site 1
Second test on second die	PTR/MPR/FTR, head 1 site 2
Second test on third die	PTR/MPR/FTR, head 2 site 1
Second test on fourth die	PTR/MPR/FTR, head 2 site 2
Repeat test suite on all four dice	PTR/MPR/FTR
	...
	PTR/MPR/FTR
Test suite finishes first on second die; this PRR has results of all tests on this die.	PRR, head 1 site 2
Finish test suite on remaining three dice	PTR/MPR/FTR, head 1 site 1 PTR/MPR/FTR, head 2 site 1 PTR/MPR/FTR, head 2 site 2
Results of all tests on first die	PRR, head 1 site 1
Results of all tests on third die	PRR, head 2 site 1
Results of all tests on fourth die	PRR, head 2 site 2

Repeat sequence for next set of four dice	PIR
	...
	PRR
	...
Information for final four dice	PIR, head 1 site 1
	PIR, head 1 site 2
	PIR, head 2 site 1
	PIR, head 2 site 2
Perform each test on each die	PTR/MPR/FTR
	...
	PTR/MPR/FTR
	...
Test suite finishes on first and second dice; all tests now complete on head 1 dice	PRR, head 1 site 1
	PRR, head 1 site 2
Test results for all dice of head 1 wafer	WRR, head 1
Perform remaining tests on head 2 dice	PTR/MPR/FTR, head 2 site 1
	PTR/MPR/FTR, head 2 site 2 ...
Test suite finishes on third and fourth dice; all tests now complete on head 2 dice	PRR, head 2 site 1
	PRR, head 2 site 2
Test results for all dice of head 2 wafer	WRR, head 2
Testing remaining wafers:	
Repeat sequence for each set of wafers	WIR, head 1
	WIR, head 2
	...
Test results for all dice of next wafers	WRR, head 1
	WRR, head 2
Final results for entire lot:	
Summary (count of test executions, count of failed parts, etc.) for each test in job plan (one TSR per test in plan)	TSR
	...
	TSR
Count of dice placed in each hardware bin (one HBR per hardware bin)	HBR
	...
	HBR

Count of dice assigned to each logical bin (one SBR per software bin)	SBR
	...
	SBR
Part count for head 1, site 1	PCR
Part count for head 1, site 2	PCR
Part count for head 2, site 1	PCR
Part count for head 2, site 2	PCR
Part count for entire lot (HEAD_NUM = 255)	PCR
Global summary of results for entire lot	MRR

< >	
	2015 Teradyne, Inc. All rights reserved.

strecstru.2.8

<

>

STDF Record Structure : Storing Repair Information

Storing Repair Information

Data for repair of memory, PC boards, and other parts can be stored and passed between the test and repair processes using the STDF format. Repair information for each part tested is stored in the PART_FIX field in the Part Results Record (PRR).

You can keep repair data in the same STDF file as all other test information or separate it to minimize the number of bytes passed from one process to the next. The following examples show how STDF records are used to store repair information. Additional STDF records can be used in the file for more information as desired.

The following is the ordering of the minimum records required for an STDF file containing memory repair data from wafer probe:

File Ordering for Wafer Probe Repair Data

Information Datalogged	Record Type Used
File Attributes Record	FAR
Master Information Record	MIR
Wafer information for 1st wafer	WIR
Part results for 1st device on wafer	PRR
Part results for 2nd device on wafer	PRR
	...
Part results for last device on wafer	PRR
Wafer results for 1st wafer	WRR
	...

Wafer information for last wafer	WIR
Part results for 1st device on wafer	PRR
Part results for 2nd device on wafer	PRR
	...
Part results for last device on wafer	PRR
Wafer results for last wafer	WRR
Part Count Record	PCR
Master Results Record	MRR

The following is the ordering of the minimum records required for an STDF file containing PC board repair data from a board tester:

File Ordering for Board Repair Data

Information Datalogged	Record Type Used
File Attributes Record	FAR
Master Information Record	MIR
Part results for 1st PC board	PRR
Part results for 2nd PC board	PRR
	...
Part results for last PC board	PRR
Part Count Record	PCR
Master Results Record	MRR

<

>

strecstru.2.9

	
---	---

STDF Record Structure : Using Pin Mapping Records

Using Pin Mapping Records

When testing devices, either packaged or as part of a wafer, device pins are mapped to tester channels. This mapping is defined in the Pin Map Record (PMR). Each channel has a type and a name. Each pin has a physical and a logical name. The PMR defines one unique association between a channel and a pin and assigns that mapping a number, known as the PMR Index. These indexes are in the range 1 -32,767.

Pins are sometimes defined in groups, such as address pins or data pins. The Pin Group Record (PGR) allows a group of pins to be named and given a group index number. The PGR lists PMR Indexes for pins in a pin group and assigns a name and a Group Index number to that group. Group index numbers are in the range 32-768 - 65,535.

A pin group includes a display radix and an operating mode. Pin groups that are multiplexed (that is, that serve multiple functions at different times) may have more than one set of radices and operating modes. Depending on the tester type and device type, those modes may also have different data representations. The Pin List Record (PLR) defines a mapping between one or more pins (by pin index) or pin groups (by group index) and their corresponding display radices and operating modes. It also defines the programmed-state and returned-state character representations for those pins.

Both the Functional Test Record (FTR) and the Multiple-Result Parametric Record (MPR) use pin indexes defined in the PMR to associate state information with their corresponding pins. Both programmed states and returned states can be decoded and displayed using information from PMRs, PGRs, and PLRs associated with the pin indexes listed in FTRs and MPRs.

Software that decodes and displays functional test data will use the pin index mapping in FTRs and MPRs to determine which pins had what values. The software can use the PMR to determine the physical and logical name of the pin, the channel name and type associated with that pin, and which test head and site that channel is connected to. Using data from the PGR, the software can

determine whether a pin is part of a pin group and, if so, what the group name is and what other pins are part of that group. Data from the PLR can then tell the software how data associated with that pin should be displayed to make it understandable to engineers and programmers dealing with that type of tester and device.

<		>	
	2015 Teradyne, Inc. All rights reserved.		